

从透明性到可执行责任：政务算法治理的责任锚定机制与可查询审计

目标期刊：《电子政务》（主投）；《中国行政管理》（备选）

类型：数字治理 / 公共治理 / 信息治理（机制设计）

状态：中文投稿版 v0.2

作者：付毅

作者单位：川北医学院后勤管理处，四川 南充 637000

邮箱：fuyi@nsmc.edu.cn

摘要

在政务数字化与算法应用深化背景下，算法规则、模型与参数的频繁上线与变更，使治理难题从“能否解释算法”转向“能否对高影响决策形成可追溯、可审计的责任链条”。仅依赖透明性/可解释性，往往难以覆盖跨部门协同、商业敏感、黑箱模型与高频迭代等现实约束。本文提出“可执行责任”框架：将责任从文本条款与事后追责，落实为可操作的制度—技术组合，包括责任签章（授权主体签署）、约束集注册（标准版本化与引用）、安全底座/断路器（关键节点强制拦截与降级）、以及审计记录（决策—标准—证据—时间绑定）。在此基础上，本文给出机制流程图与制度要素最小集，并以健康码赋码规则上线/变更为情景化案例，说明如何用“可查询审计”替代事后叙事争辩，提升政务算法治理的可问责性与可落地性。

关键词：数字治理；算法治理；可执行责任；责任锚定；可查询审计；政务信息化

1. 引言：算法治理的“西西弗斯神话”

近年来，各国政府出台了大量算法监管政策（如《互联网信息服务算法推荐管理规定》）。然而，监管部门面临着巨大的执法难题：

- 看不懂：算法代码极其复杂，且实时变动。
- 管不住：即使发现了问题，往往已造成了社会后果（如大数据杀熟、外卖骑手困境）。我们似乎在推着一块叫“合规”的巨石上山，但每当算法模型更新一次，巨石就滚落下来。

2. 困境分析：透明性的幻象

主流观点认为，只要算法公开透明，问题就解决了。

这是一个幻象。

- 技术层面的不可知：神经网络的参数是数以亿计的浮点数，公开了也无人能懂。

- 商业层面的博弈: 算法是企业的核心资产，强制公开面临巨大的法律阻力。
更根本的是，透明不等于负责。我知道你为什么杀熟（因为追求利润最大化），但这并不能阻止你杀熟。

3. 理论突破：从“解释”到“约束”

我们需要从“认识论治理”（试图理解算法为什么做出某个结果）转向“制度—技术协同治理”（把治理要求编译为系统可执行的约束与拦截点）。治理的目标不是要求每个决策都“可被解释”，而是让关键决策在制度上可被授权、可被约束、可被记录、可被审计。

本文提出“可执行责任”（Executable Responsibility）：

- 责任不应只存在于文本条款与事后追责中，而应落实为系统架构中的控制点（如授权签署、约束引用、断路器拦截、审计记录）。
- 可执行不等同于全自动化，而是把“谁能做决定、必须引用什么标准、需要留下什么证据”固化为可操作流程与记录结构。

4. 机制设计：可执行责任框架（责任签章 + 约束集 + 安全底座 + 审计）

4.1 机制流程（图1）

为便于政务场景落地，本文把“可执行责任”抽象为一个可查询审计的闭环流程。



图1 可执行责任机制流程（建议配套矢量图）：见
submission/figure1_mermaid.txt。

4.2 制度要素表（表1）

为避免“口号式治理”，本节给出最小制度要素，便于在政务平台、监管沙箱或行业监管接口中实现。

表1 可执行责任的制度要素（最小集）

要素	定义	记录/约束要求	典型落点
责任主体	具备授权的自然人/机构角色	角色与授权边界可追溯	账号体系/授权矩阵
责任签章	对上线/变更的制度性签署	“谁在何授权下签署了什么版本”可查询	上线审批/变更单
约束集	可执行或可核验的标准集合（版本化）	“无引用不放行”；版本变更留痕	规则库/政策即代码
安全底座/断路器	关键节点的强制拦截与降级	违规即阻断/降级；保留拦截证据	SDK/网关/中台
证据摘要	对测试/评估/核验材料的摘要承诺	可抽样复核；防事后补证	报告摘要/哈希承诺
审计记录	决策—标准—证据—时间的绑定记录	追加写入、不可随意改写	审计日志/账本
例外与申诉	例外放行与纠错通道	TTL + 补救义务 + 复盘	例外流程/申诉工单

4.3 典型应用：政务算法规则与模型上线

在政务信息系统中，常见的“高影响上线/变更”包括：规则参数调整、模型版本升级、阈值策略更新、跨部门数据共享策略变更等。可执行责任框架的直接价值在于把“放行”变成可审计对象，从而让监管与问责从“解释争辩”转向“记录审计”。

5. 本土化案例（政务语境表述）：以健康码/赋码规则为例

以“健康码/赋码规则上线与变更”为例，其治理困境在于：判定规则可能频繁调整、跨部门协同复杂、公众申诉成本高。若采用可执行责任：

- 1. 赋码/变更即负责：每一次赋红码或规则变更，系统必须记录引用的约束集版本、授权主体与证据摘要。
- 2. 自动解除与例外：对解除条件形成可执行约束；对例外放行设置 TTL 与补救义务，并形成可查询记录。

3. 可查询审计：监管可通过查询定位“谁在何标准下批准了哪次规则变更”，减少事后争辩与责任转嫁。

6. 讨论与局限

1. 落地成本与组织摩擦：可执行责任会引入审批、证据生成与记录管理成本。建议采取风险分级：对高影响变更强制事件化，对低风险变更抽样审计。
2. 形式主义风险：若将“有记录”误当作“已负责”，可能产生合规剧场。应配套抽样复核、例外频率审计与证据复现审计。
3. 隐私与最小必要记录：政务场景记录结构应坚持最小必要原则，明确哪些字段用于问责、哪些字段属于敏感信息并采取脱敏与受控访问。

7. 结论

算法治理的出路，不在于把系统变得更“可解释”，而在于把责任变得可执行、可记录、可审计。本文提出的“可执行责任”框架以责任签章、约束集注册、安全底座与断路器拦截、以及审计记录为最小组合，能够在不强制单一技术路线的前提下，把政务场景中高影响决策的放行流程转化为可查询的责任链条，为“事前可约束、事中可拦截、事后可审计”的政务算法治理提供一种可操作路径。

参考文献（候选，待按期刊格式统一）

- [1] 国家互联网信息办公室. 互联网信息服务算法推荐管理规定[Z]. 2022.
- [2] 中华人民共和国个人信息保护法[Z]. 2021.
- [3] Diakopoulos N. Accountability in algorithmic decision making[J]. Communications of the ACM, 2016, 59(2): 56–62.
- [4] Kroll J A, Huey J, Barocas S, et al. Accountable algorithms[J]. University of Pennsylvania Law Review, 2017, 165(3): 633–705.
- [5] Raji I D, Smart A, White R N, et al. Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing[C]//FAT*. 2020.
- [6] Power M. The Audit Society: Rituals of Verification[M]. Oxford University Press, 1997.
- [7] Souppaya M, Scarfone K, Dodson D. Secure Software Development Framework (SSDF) Version 1.1[R]. NIST SP 800–218, 2022.